



Universidad Nacional de Río Cuarto

Facultad de Ciencias Exactas Físico Químicas y Naturales

Departamento de Computación

Taller de Diseño de Software

Prof. Francisco Bavera

Compilador para el lenguaje C-TDS

“Entrega Etapa 1 — Análisis Léxico y Sintáctico”

Emiliano Bernal

Agustín Cesari

Samuel Quintero

Septiembre 2026

1. Decisiones de diseño

1.1. Conflictos *shift/reduce* y *reduce/reduce* en `var_decl` y `method_decl`

La gramática inicial producía conflictos *shift/reduce* y *reduce/reduce* al definir `var_decl` y `method_decl`, ya que ambas producciones comparten el prefijo `type id` y el parser LALR(1) debe decidir, con un único token de anticipación (*lookahead*), qué regla aplicar.

Concretamente, la gramática inicial definía las declaraciones de variables y de métodos como dos listas recursivas separadas, una a continuación de la otra:

```

1 program: var_decls method_decls;
2
3 var_decls: %empty | var_decls var_decl;
4
5 method_decls: %empty | method_decls method_decl;
6
7 var_decl: type ID SEMI;
8
9 method_decl: type ID LPAREN params RPAREN block
10             | VOID ID LPAREN params RPAREN block
11             ;

```

Detectamos esta situación al intentar compilar `parser.y` con el siguiente ejemplo:

```

1 int main(){
2     int x;
3     int y;
4     x = 1;
5     y = 1;
6     x = x + 3 * 2 * y;
7     y = 2 + 2;
8     return;
9 }

```

1.1.1. Resolución del conflicto

Para resolverlo, se unificó la gramática en una única lista recursiva que admite `var_decl` y `method_decl` en cualquier orden:

```

1 program: %empty
2         | program var_decl
3         | program method_decl
4         ;

```

Esta unificación elimina la ambigüedad estructural: al no forzar un orden estricto entre declaraciones de variables y declaraciones de métodos, el parser deja de tener que decidir entre dos producciones que comienzan con el mismo prefijo en el mismo contexto, y los conflictos desaparecen.